Master CTF Methodology — works on ANY box

ISEC6005 · the universal framework: recon → route to the right technique → escalate · for processes you have NOT practised

The one idea that never changes

- You do NOT memorise boxes. You **enumerate every service, let each finding point to the next step, and read evidence (errors, headers, files) carefully.**
- Every flag — even a technique you've never seen — is reachable by the same loop: **find a service → identify exactly what it is → apply that service's playbook → if stuck, research the exact version/product and test the obvious.**
- "The answer is often obvious. Do not overthink the scenario."

1. The universal loop (memorise this)

```
START
  |
  v
[1] RECON ........ full TCP scan, identify EVERY open port + service + version
  |
  v
[2] FOR EACH SERVICE ... open the matching playbook (routing table, Section 2)
  |                       enumerate it (anonymous first, least-intrusive first)
  v
[3] READ EVERYTHING ... errors, headers, files, notes -> they name the next step
  |
  v
[4] EASY FLAG ........ a file you can read / a value disclosed. Capture it.
  |                    (keep every credential/clue you saw -- they feed intermediate)
  v
[5] INTERMEDIATE ..... reuse a found credential, OR exploit the service
  |                    (login, webshell, SQLi, upload) -> foothold / RCE
  v
[6] HARD ............. privilege escalation -> SYSTEM/root -> read protected flag
  |
  v
[7] DOCUMENT ......... commands, evidence, cause/trust/blast/impact/mitigation
```

2. Service routing table (port → what to try)

After recon, match each open port to its technique. This covers the common services and points you to the right reference. Unusual ports still map by their **service/version**, not the number.

Port(s)	Service	First things to try	Reference
21	FTP	anonymous login; read all files; check writable; non-std port	FTP guide
22	SSH	login once you have a credential (crack/find it first)	—
23	Telnet	banner; default/anonymous creds	—
25/110/143	Mail	banner; user enumeration (VRFY)	—
53	DNS	zone transfer (axfr); subdomain info	—
80/443/8080/8000/9999/242	HTTP(S)	follow links; gobuster; directory listing; robots.txt; source	Web guide
111/2049	NFS	showmount -e; mount the export; read files	—
139/445	SMB	anonymous share list; read custom share; enum4linux	SMB guide

161	SNMP	snmpwalk with 'public' community string	—
1433/3306/5432	DB (MSSQL/MySQL/Postgres)	default creds; if web-facing, SQLi	SQLi guide
3389	RDP	login with found creds (xfreerdp)	—
5985/5986	WinRM	login with found creds (evil-winrm)	—
8082 / other app	App console (e.g. H2, Jenkins, Tomcat)	default creds; known-CVE RCE (searchsploit)	Web guide

3. Step 1 in detail — recon that misses nothing

```
# full TCP port sweep (find EVERY open port, incl. weird high ports)
sudo nmap -p- --open <IP>

# then fingerprint just the open ports (service + version + default scripts)
sudo nmap -sC -sV -p <comma,separated,open,ports> <IP>

# optional UDP top ports (DNS/SNMP/TFTP live here)
sudo nmap -sU --top-ports 20 <IP>
```

Rules for this range

- Never use `-T5` (lockout). Reverse shells only on ports **4444 / 5555 / 6666**. Metasploit may be used **once only** — prefer manual methods (more marks).

Why identify the VERSION, not just the port

- The exact product + version is what lets you handle an UNSEEN service: you search it, find default creds or a known exploit, and confirm the version matches before trying it.

4. When you hit a process you have NOT learned

This is the real skill the exam tests. Work through this checklist on any unfamiliar service — it resolves most 'new' methods:

Ask	How to check	What it usually unlocks
What EXACTLY is it?	nmap -sV version + the http-title / banner	names the product to research
Does it allow anonymous / default login?	try blank/guest/admin:admin, sa:(blank), anonymous	instant access (very common for Easy/Intermediate)
Is there a known exploit?	searchsploit <product> <version>	a documented RCE to adapt (confirm version first)
Does it leak info?	curl -i (headers), error messages, /robots.txt, banners	paths, versions, usernames, file locations
Can I read or write files?	test a harmless read, then a harmless write/marker	file disclosure, or a webshell/upload foothold
Did an earlier service give a credential?	reuse it here (SSH/WinRM/RDP/DB/app login)	the intended intermediate path

The research move (allowed: Google AI Mode for assistance, not the answer)

- Take the exact product + version string from nmap and search it with the words **default credentials**, **exploit**, or **CTF**.
- **searchsploit <product>** queries the local Exploit-DB offline — no internet needed. Read the exploit, confirm your version matches, adapt the IP/port/payload.
- Always **confirm the version matches** before running an exploit — an exploit for a different version just wastes time or crashes the service.

5. The evidence-driven mindset (why it always works)

• **Every finding points to the next.** An FTP note names a web header; an SMB share holds WordPress creds; a SQL error leaks a file path. Follow the trail — don't guess.

• **Anonymous / default first.** The easiest win is almost always an open door: anonymous FTP/SMB, directory listing, default app creds. Try these before anything clever.

• **Read errors and headers.** They hand you database type, file paths, usernames, and versions for free (`curl -i`, SQL error messages, service banners).

• **Keep every credential and clue.** The Easy phase routinely contains the key to the Intermediate (a password, a policy, a login path).

• **Prove, don't assume.** Test a harmless marker before a real payload; separate 'I demonstrated this' from 'this might be possible'.

6. Escalating: Easy → Intermediate → Hard (general patterns)

Stage	Typical pattern	Common techniques
Easy	Read a file that's exposed with no auth	anon FTP/SMB, directory listing, follow links, status/API pages, default-cred page
Intermediate	Turn access into a foothold / RCE as a low user	reuse found creds (SSH/WinRM), SQLi (read creds or write webshell), upload a v
Hard	Escalate the low user to SYSTEM/Administrator	misconfig/service/privilege abuse, stored creds (KeePass/config), known local-pri

Credential handling you'll reuse a lot

• Hash found? Identify by shape/prefix, crack with the right hashcat mode (32-hex Windows = NTLM `-m 1000`; `$apr1$` = `-m 1600`; `$6$` = `-m 1800`) against rockyou (top-250 = fast).

• Then log in through whichever port is open: SSH (22), WinRM (5985, `evil-winrm`), RDP (3389, `xfreerdp`).

7. Read-the-flag commands (any shell)

```
# Windows (cmd): find and read the flag
dir /s /b C:\Users\*flag*.txt
type C:\Users\<user>\Desktop\<flag>.txt
# Windows (PowerShell, forward slashes avoid escaping issues):
powershell -c cat C:/Users/<user>/Desktop/<flag>.txt

# Linux:
find / -name '*flag*' 2>/dev/null
cat /home/<user>/<flag>.txt
```

Core idea: the method is universal — enumerate every service, apply its playbook, read evidence, and research the exact product when it's new. The specific box never matters; the loop does. Use the per-service references (FTP, SMB, Web, SQLi) for the detail of each technique.